

8.12模拟面试

1.connect五个参数？

AutoConnection，默认值，如果信号发送者和接收者在同一线程，则用
DirectConnection，否则用QueueConnection

DirectConnection：槽函数在信号发送时直接被调用，这在多线程环境下比较危险

QueueConnection：槽函数被放入事件循环，多线程一般使用这个

BlockingQueueConnection：调用时机和QueueConnction一致，但是信号发出者发出信号后会等待槽函数执行完再结束，常用于多线程同步

UniqueConnection：同样的信号与槽连接只计入一次，前面几种可以按位或进行结合使用

2.信号和槽的底层？映射链表存储到哪？

3.signal和slot修饰符的作用

首先信号和槽使用前提是类继承自QObject，内部有Q_OBJECT宏，moc识别后为所有符合条件的类生成一个moc文件，并把该类中信号和槽相关信息打包到moc文件中，在调用connect函数时，把对应的信号和槽在连接表中建立连接（单独文件打包），在调用emit时，根据映射表中连接信息，找到对应的槽函数并返回调用

```
1. #define signals public
2. #define slots /* nothing */
```

signal和slot只是一个宏定义，看起来好像没什么，但为什么moc可以识别，因为宏替换发生在预处理期，此时moc会先识别Q_OBJECT宏，然后在替换signal和slot时提取信号和槽的签名，记录emit关键字（也是宏，通常替换为空），标记信号发出点，并把这些信息打包放在元信息结构体中（staticMetaObject, 里面具体有连接信息和映射表之类的）

4.UI绘制除了QML，还可以使用什么绘制？

QML语言通过代码生成控件，支持动态布局（类似HTML，CSS,JS），适合动态动画，由于代码生成，可以和OpenGL，Vulkan等结合吃到硬件加速

此外，我们还可以用Qt Designer中的控件加上代码绘制想要的控件，这样的话只能基于CPU绘制，效率比较低

5.为什么UI只能在主线程中操作

假如子线程有权限操作控件，可能会导致冲突，而通过信号槽转发到主线程进行处理，由于走的是事件分发，且主线程具备唯一性，此时不会出现并发问题

6.C语言和C++的静态变量有什么区别？

c++多了类的概念，所以有类内成员静态变量和类内静态函数，都属于类，操作时通过类名作用域操作，其中静态成员可以在类外直接初始化（编译器），函数可以直接用类名+作用域调用，但函数无权操作非静态成员

7.C++三大特性怎么在C上实现？

封装：由于c中struct默认访问修饰符为public，没有protected和private限制，所以要做到这个私有和保护的效果，可以让成员变量在结构体中声明，对外提供访问内部变量的接口函数（类似于单例那种）

继承：父类的结构体成员在子类最前面定义

多态：父类的函数定义为函数指针，子类重写某个函数时，先判断父类中有没有可以指向该函数的函数指针，如果有就把父类函数指针指向子类对象

8.面向过程和面向对象编程思想？

9.STL六大组件介绍一下？

容器：封装数据和结构的集合（序列，关联，容器适配器）

迭代器：访问容器内部数据的接口，避免直接和容器解除，实现了解耦（输入输出，前向，双向，随机）

适配器：对原有的组件进行修饰，赋予其新的功能（比如逆序迭代器就是迭代器适配器，容器适配器由deque修改得到）

分配器：分配和回收空间

算法：实现某些功能而定义的特定方法，本质是模版函数（比如sort，find，for_each）

仿函数：重载operator()，让类变成可调用对象，实现一些特定的功能

10.分配器的缓存机制？

分配器用于分配空间，缓存机制就是类似于malloc底层那样的内存池，回收后空间交给分配器统一管理，而不是返还给操作系统

11.vector和list的区别?

vector随机访问, list顺序访问

vector底层一片连续空间，list是双向循环链表

vector插入删除需要移动空间（且导致后面的所有元素迭代器失效，扩容全失效），list只删除当前元素（只影响当前元素迭代器）

vector动态管理和分配内存，会预分配一大片空间，list每次新节点创建都要分配空间

vector中只存对应数据，list还有指针域消耗空间更大

12.vector底层有几个指针?

三个, `_start`, `_finish`, `_end_of_storage` , 分别指向vector开始, 结束元素, 和容量末尾元素位置

13.怎么回收vector剩余空闲的空间?

14.resize和reserse区别?

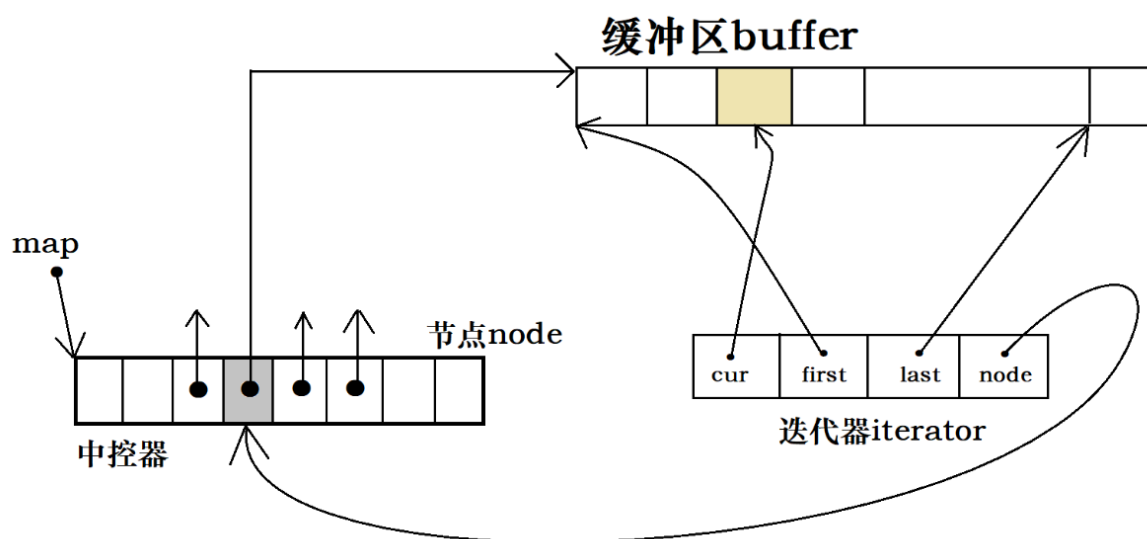
resize只影响size, 不影响容量, reserve影响容量, 但改变空间大小时, 本质是分配新的, 迁移元素, 销毁旧的, 有额外开销

如果只想回收剩余空间，可以用shrink to fit函数

15.怎么实现一个vector? 泛型编程实现

16.什么时候扩容1.5或者2，为什么要这么设计？

17.deque对应迭代器有几个指针?



具体deque结构不再细讲，迭代器主要有四个指针，cur表示当前访问元素，first表示当前内存块起始位置，last表示结束位置，node表示对应节点在中控器中的地址位置

18.default和delete的作用？

default表示使用编译器默认的函数，delete表示禁用对应函数

19.一个空类中有什么默认的函数？有哪些函数显式声明后就不会自动生成？三/五法则？

构造，析构，移动构造，移动赋值，拷贝构造，拷贝赋值

其中，构造的三个，普通构造，移动构造，拷贝构造，显式声明一个其余两个不会再生成（因为参数可能一致，造成歧义）

拷贝赋值和移动赋值也是互斥的

三五法则是对于管理资源的一种约束，其中三法则是对于拷贝这一块，由于编译器默认给出函数是浅拷贝，要自己手动实现则用深拷贝（三法则对应析构，拷贝构造，拷贝赋值），五法则则是在引入右值引用后的进一步约束，要求我们管理资源必须要手动定义五种函数（三法则基础上加上移动构造和移动赋值）

20.介绍智能指针？&ptr，&&ptr，ptr，unique_ptr能接收哪种参数？

只能接收右值类型的指针

21.lambda表达式有什么问题？怎么解决？本质是什么？

值捕获和引用捕获同时发生，捕获函数超过声明周期导致悬空，捕获const修饰的变量和类

lambda本质是个闭包，通过重载operator()变成一个可调用对象，不能对lambda函数体中的变量进行修改，除非用mutable修饰

22.介绍了解的设计原则？为什么合成复用原则用组合代替继承？

通过组合而不是继承，降低了类与类之间的耦合度，且父子类成员可能冲突，此时可以更明确修改其中一方，而不是覆盖

23.单例模式的几种创建方式？线程不安全的是哪个？怎么解决？双重检查锁每个if的作用？

24.指令重排问题？volatile，one_call？

懒汉饿汉，其中懒汉不安全，可以用双检索，双检索可能因为编译器指令重拍出问题，此时彻底解决要用call_once

双检索中，第一个if判断是否变量有空间，如果有就直接用了，没有则进入函数中，此时进行上锁操作，由于并发，所以只有一个线程能进行单例的生成，释放锁后判断第二个if，此时再检查变量是否有空间，有则使用

25.介绍一下中介者模式？优点缺点？

中介者和其他成员交互，作为中间件，降低了耦合度，有利于维护和拓展，但中间件压力过大，可能影响其性能

26.观察者模式有用到吗？比如信号和槽，好友列表刷新

观察者模式由一对多（一个被观察者，多个观察者），被观察者中有所有观察者的一个信息，当被观察者产生相应事件时，通过分发事件给观察者，观察者对此做出相应

观察者模式由于通知观察者时顺序不可控所以可能出问题，以及被观察者挨个通知，需要额外性能开销

具体场景有信号槽机制，redis订阅发布机制，哨兵机制等